Failure Report

Signup Bypasses create_workspace

Breaking Collection Tests

Project: APICraft -- API Tester

Test file: backend/tests/test_collections.py

Date: 2026-06-16

1. The Symptom

Two tests failed on the first run of test_collections.py:

```
FAILED tests/test_collections.py::TestListCollections::test_returns_all_collections
FAILED tests/test_collections.py::TestDeleteCollection::test_deleted_collection_not_in_list
```

Failure 1 output

```
assert len(cols) == 3
AssertionError: assert 2 == 3
```

Failure 2 output

```
assert col_id not in col_ids
AssertionError: assert 1 not in [1]
```

Both tests operated on the default workspace created during signup and expected a default collection to already exist inside it. Neither had ever called the collections list endpoint before performing write operations.

2. Root Cause -- signup_user Creates the Workspace Directly

The auth_service.signup_user function creates the default workspace by instantiating the Workspace model directly and committing it. It never calls workspace_service.create_workspace, which is the only other path that seeds a default collection.

backend/services/auth_service.py

```
def signup_user(username, first_name, email, password):
    # ...
```

```
    user = User(username=username, first_name=first_name, ...)
    db.session.add(user)
    db.session.flush()

    # Workspace created directly -- create_workspace() is never called
    default_workspace = Workspace(
        name=f"{first_name}'s Space", user_id=user.id, is_default=True
    )
    db.session.add(default_workspace)
    db.session.commit()   # no ensure_default_collection() here
    return user, None
```

Compare this to workspace_service.create_workspace, which seeds the default collection immediately via ensure_default_collection:

backend/services/workspace_service.py

```
def create_workspace(user_id, name):
    workspace = Workspace(name=name, user_id=user_id)
    db.session.add(workspace)
    db.session.flush()
    ensure_default_collection(workspace.id)  # <- never reached from signup
    return {'id': workspace.id, 'name': workspace.name, ...}
```

Because signup_user skips this entirely, the default workspace is created with zero collections. The default collection ("My Collection") is only seeded lazily -- the first time GET /api/workspaces/{id}/collections is called:

backend/services/collection_service.py

```
def get_collections_by_workspace(workspace_id):
    ensure_default_collection(workspace_id)  # <- seeding only happens here
    collections = Collection.query.filter_by(workspace_id=workspace_id).all()
    return [_serialize(c) for c in collections]

def ensure_default_collection(workspace_id):
    exists = Collection.query.filter_by(workspace_id=workspace_id).first()
    if not exists:
        collection = Collection(
            name='My Collection', workspace_id=workspace_id, is_default=True
        )
        db.session.add(collection)
        db.session.flush()
        db.session.add(Request(name='Get data',  method='GET',  ...))
        db.session.add(Request(name='Post data', method='POST', ...))
```

```
        db.session.commit()
```

3. Why Each Test Failed

test_returns_all_collections

The test assumed the default collection already existed after signup and expected 3 total collections after adding 2 more:

```
def test_returns_all_collections(self, client, auth_data):
    ws_id = auth_data['default_workspace_id']
    client.post(f'/api/workspaces/{ws_id}/collections')  # ID=1, is_default=False
    client.post(f'/api/workspaces/{ws_id}/collections')  # ID=2, is_default=False
    cols = client.get(f'/api/workspaces/{ws_id}/collections').get_json()
    assert len(cols) == 3  # FAILS -- only 2 exist, default was never seeded
```

The final GET calls ensure_default_collection, but since 2 collections already exist, the guard "if not exists" prevents any new seeding. The result is 2, not 3.

test_deleted_collection_not_in_list -- compounded by SQLite ID reuse

This failure had a second layer. After deleting the only collection from an unseeded workspace, the table became empty. ensure_default_collection then auto-created a fresh default collection -- and SQLite's INTEGER PRIMARY KEY (without AUTOINCREMENT) reuses IDs when the table is empty, assigning the new default collection the same ID=1 as the deleted one.

```
def test_deleted_collection_not_in_list(self, client, auth_data):
    ws_id = auth_data['default_workspace_id']
    col_id = self._create_non_default(...)["id"]  # ID=1 (first in empty table)
    client.delete(f'/api/collections/{col_id}')   # deletes ID=1
    col_ids = [c["id"] for c in client.get(...).get_json()]
    # ensure_default_collection fires, creates new default -- gets ID=1 again!
    assert col_id not in col_ids  # FAILS: assert 1 not in [1]
```

4. How the Agent Diagnosed It

After seeing both failures trace back to auth_data['default_workspace_id'], the agent read auth_service.py directly. Tracing the signup flow confirmed that Workspace is instantiated without ever calling create_workspace or ensure_default_collection.

Cross-referencing workspace_service.create_workspace showed that seeding only happens through that function. Since signup bypasses it, the workspace is created empty and the seeding is entirely deferred to the first GET.

The SQLite ID reuse was inferred directly from the error output: "assert 1 not in [1]". A deleted ID reappearing in a fresh insert is only possible when the table is empty and SQLite reassigns the lowest available integer. This confirmed the table had been fully emptied before the re-seeding occurred.

5. The Fix

Both tests were corrected by adding one GET call before any write operations. This forces ensure_default_collection to run, occupying ID=1 before any non-default collections are created. All subsequent inserts receive higher IDs and no collision is possible.

test_returns_all_collections -- fixed

```
def test_returns_all_collections(self, client, auth_data):
    ws_id = auth_data['default_workspace_id']
    client.get(f'/api/workspaces/{ws_id}/collections')  # seeds default (ID=1)
    client.post(f'/api/workspaces/{ws_id}/collections') # ID=2
    client.post(f'/api/workspaces/{ws_id}/collections') # ID=3
    cols = client.get(f'/api/workspaces/{ws_id}/collections').get_json()
    assert len(cols) == 3  # PASSES
```

test_deleted_collection_not_in_list -- fixed

```
def test_deleted_collection_not_in_list(self, client, auth_data):
    ws_id = auth_data['default_workspace_id']
    client.get(f'/api/workspaces/{ws_id}/collections')  # seeds default (ID=1)
    col_id = self._create_non_default(...)["id"]  # ID=2
    client.delete(f'/api/collections/{col_id}')   # deletes ID=2
    col_ids = [c["id"] for c in client.get(...).get_json()]
    assert col_id not in col_ids  # PASSES: assert 2 not in [1]
```

6. Design Inconsistency Exposed by the Failure

The failure revealed that two workspace creation paths behave differently. Any code that assumes a workspace always has a collection immediately after creation (e.g. a background job, import feature, or API consumer) would find an empty workspace if it was created through signup.

Creation path	Seeds default collection?
POST /api/workspaces -> workspace_service.create_work	Yes -- immediately
Signup -> auth_service.signup_user()	No -- only on first GET /collections

The tests made this implicit assumption visible and permanently documented. Any future developer reading these tests will see exactly why the GET call must precede write operations in this context.